Lecture 12: Storage I — I/O, File Descriptors & read/write

Session 15 · Mon Oct 19, 2026

Contents

Recap: Lectures 9 and 10 . . . 1
Outline . . . 1
Everything Is a File . . . 2
Run It: A Process's Open Descriptors . . . 2
File Descriptor Table . . . 2
Basic I/O . . . 3
A Complete Loop: cat . . . 3
Run It: cat in Four System Calls . . . 3
A Short Read Is Not an Error . . . 4
Run It: Short Reads from a Pipe . . . 4
Robust Read . . . 4
Why the `EINTR` Case Is Not Optional . . . 5
File Position . . . 5
open() Flags . . . 5
The Shell's Redirections Are open() Flags . . . 5
File Metadata: stat . . . 6
Buffered vs. Unbuffered I/O . . . 6
Run It: One Byte per System Call . . . 6
AI Systems Connection . . . 7
The Lecture in One Slide . . . 7
Practice . . . 7
Answers to Practice . . . 7
Reading . . . 8

Recap: Lectures 9 and 10

Lecture 9. A program asks the kernel for anything it cannot do alone through a **system call**. A system call costs as much as hundreds of function calls, so a program should do as much work as it can per call.

Lecture 10. `mmap` can attach a file to a range of addresses, and the program then reads the file through a pointer, with no `read` call.

Today. Most programs do not map their files. They ask the kernel to copy bytes in and out with `read` and `write`, and they name the file with a small integer called a **file descriptor**. Every slide today is about those two calls and that integer.

Outline

What you will learn today:

1. What a file descriptor is, and what the kernel keeps behind it
2. The four basic calls: `open`, `read`, `write` and `close`
3. Why `read` can return fewer bytes than you asked for, and how to loop

4. What `EINTR` means and why a loop must retry on it
5. How the file position moves, and how `lseek` moves it
6. Which `open` flags the shell uses for `>`, `>>` and `<`
7. How to read a file's size and type with `stat`
8. Why one byte per system call is slow, and how `stdio` buffers for you

Everything Is a File

Unix reaches almost everything through the same calls: regular files, directories, the terminal, pipes between processes, network sockets and devices. The program gets a **file descriptor** for each one. A file descriptor is a small non-negative integer that the kernel hands back from `open` (or `pipe`, or `socket`), and the program passes it to every later call on that file.

Every process starts with three descriptors already open:

Descriptor	Name	Usually connected to
0	`STDIN_FILENO` (standard input)	the keyboard, through the terminal
1	`STDOUT_FILENO` (standard output)	the terminal screen
2	`STDERR_FILENO` (standard error)	the terminal screen

The shell set them up before it started the program. That is why a program can print without opening anything, and why the shell can send the output to a file instead: it changes what descriptor 1 refers to.

Run It: A Process's Open Descriptors

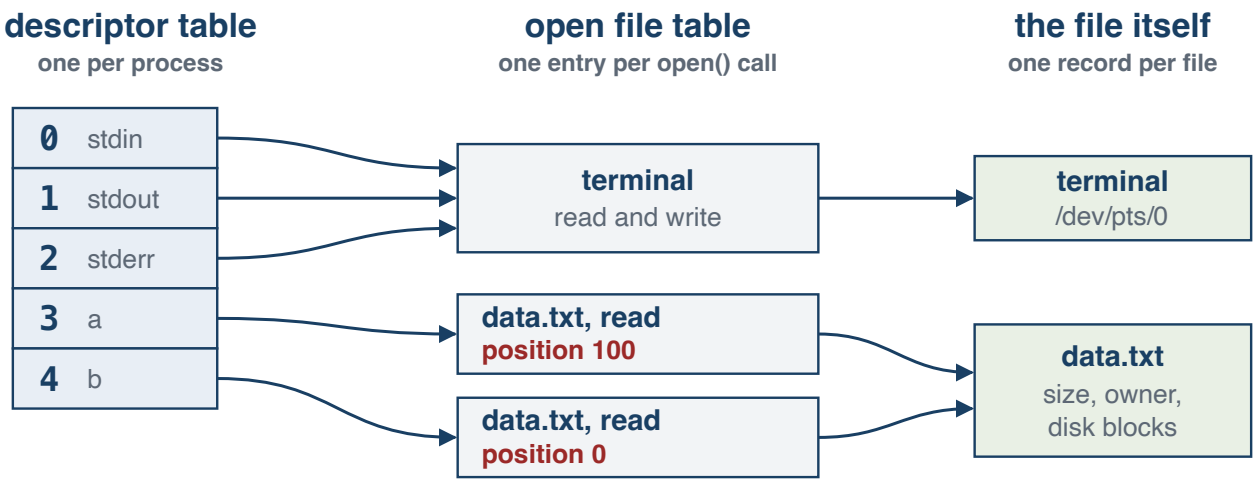
On Linux, the directory `/proc/self/fd` lists the descriptors of the process that looks at it. Here `ls` looks at its own:

```
$ ls -l -g -o /proc/self/fd                          # Linux
lrwx------. 1 64 Oct 27 14:35 0 -> /dev/pts/0
lrwx------. 1 64 Oct 27 14:35 1 -> /dev/pts/0
lrwx------. 1 64 Oct 27 14:35 2 -> /dev/pts/0
lr-x------. 1 64 Oct 27 14:35 3 -> /proc/1937167/fd
```

Descriptors 0, 1 and 2 all refer to `/dev/pts/0`, the terminal window. Descriptor 3 is the directory `ls` opened to read this list. The kernel gave it 3 because `open` always returns the **lowest unused number**, and 0, 1 and 2 were taken. (macOS has no `/proc`.)

File Descriptor Table

A descriptor is an index into the process's **descriptor table**. Each `open` call makes a new **open file table entry**, which holds the **file position**, the offset where the next `read` or `write` starts. Here a process opened `data.txt` twice, as descriptors `a` (3) and `b` (4), and then read 100 bytes through `a`:



0, 1 and 2 point to one entry, so they share one position.
a and b come from two open() calls: two entries, two positions, one file.

Basic I/O

Call	What it does	Returns
`open(path, flags)`	opens the file and makes a descriptor for it	the new descriptor, or `-1`
`read(fd, buf, n)`	copies **up to** `n` bytes from the file into `buf`	bytes copied, `0` at end of file, or `-1`
`write(fd, buf, n)`	copies up to `n` bytes from `buf` into the file	bytes copied, or `-1`
`close(fd)`	frees the descriptor so its number can be reused	`0`, or `-1`

Every one of them is a system call. On `-1`, the variable `errno` says what went wrong, and `perror` prints it as a message.

A Complete Loop: cat

`cat` opens a file, reads it a buffer at a time, and writes each buffer to standard output:

```c
int fd = open("data.txt", O_RDONLY);
if (fd < 0) { perror("data.txt"); return 1; }

char buf[4096];
ssize_t n;                                  // signed: it can hold -1
while ((n = read(fd, buf, sizeof buf)) > 0)  // stop at 0 (end) or -1 (error)
    write(STDOUT_FILENO, buf, n);           // write exactly the n bytes read
if (n < 0) perror("read");
close(fd);
```

That loop is all of `cat`. It is `01-mycat.c` in the sample code.

Run It: cat in Four System Calls

`strace` prints every system call a program makes, with its arguments and its return value (the C library's own start-up calls are left out):

```
$ printf 'hello\n' > data.txt
$ strace -e trace=openat,read,write,close ./01-mycat data.txt     # Linux
openat(AT_FDCWD, "data.txt", O_RDONLY)  = 3        # lowest unused number
read(3, "hello\n", 4096)                = 6        # asked for 4096, got 6
write(1, "hello\n", 6)                  = 6
hello
read(3, "", 4096)                       = 0        # 0 means end of file
close(3)                                = 0
```

The C library's `open` calls the system call `openat`, which does the same job. The first `read` asked for 4096 bytes and got 6, because the file only has 6. The program does not stop there. It stops when `read` returns 0.

A Short Read Is Not an Error

`read(fd, buf, n)` may return a number between 1 and `n - 1`. That is called a **short read**, and it is not a failure. It happens when:

- the file has fewer than `n` bytes left before its end;
- the descriptor is a terminal, which returns one line as soon as the user presses Enter;
- the descriptor is a pipe or a network socket, which returns whatever bytes have arrived so far.

A regular file on disk returns all `n` bytes unless it reaches the end. A program that needs exactly `n` bytes, such as a program reading a fixed-size file header, must check the count and call `read` again for the rest. `write` can be short too, on pipes and sockets, so the same rule applies to it.

Run It: Short Reads from a Pipe

`02-short-read.c` asks for 4096 bytes at a time and prints what `read` returns. Its input is a pipe from a shell command that writes `hello`, waits one second, and writes `world`:

```
$ (echo hello; sleep 1; echo world) | ./02-short-read
asked for 4096, read returned 6
asked for 4096, read returned 6
asked for 4096, read returned 0
$ ./02-short-read < 02-short-read.c
asked for 4096, read returned 491
asked for 4096, read returned 0
```

From the pipe, the first `read` returns the 6 bytes of `hello\n` at once instead of waiting for 4096. It waits a second for the next 6. It returns 0 when the shell command exits and closes its end of the pipe. From the 491-byte file, one `read` returns the whole file.

Robust Read

When a program needs exactly `n` bytes, it loops until it has them, or until the file ends:

```c
ssize_t read_full(int fd, void *buf, size_t n) {
    size_t total = 0;
    while (total < n) {
        ssize_t r = read(fd, (char *)buf + total, n - total);
        if (r < 0) {
            if (errno == EINTR) continue;   // interrupted by a signal: retry
            return -1;                      // a real error
        }
        if (r == 0) break;                  // EOF
        total += r;
    }
    return (ssize_t)total;
}
```

Each call asks only for the bytes still missing, `n - total`, and stores them after the bytes already read, at `buf + total`. The function returns `n`, or less than `n` if the file ended first. CS:APP calls this function `rio_readn`, and `errno` and `EINTR` come from `<errno.h>`.

Why the `EINTR` Case Is Not Optional

A **signal** (Lecture 10) is a message the kernel delivers to a process. Pressing Ctrl-C sends the signal `SIGINT`. A program can install a **handler**, a function the kernel runs when that signal arrives. Here is what happens when a signal arrives while `read` is waiting:

1. `read` is waiting for input, for example for the user to type a line.
2. The user presses Ctrl-C, and the kernel runs the program's handler.
3. `read` does not go back to waiting. It returns `-1` with `errno` set to `EINTR` ("interrupted system call"), and it has copied no bytes.

Nothing is wrong with the file, so the program should call `read` again. That is what the `continue` in `read_full` does. A shell is an example: it handles SIGINT so that Ctrl-C does not kill it. If it treated this `-1` as an error, every Ctrl-C at the prompt would make `fgets` return NULL, and the shell would exit as if the input had ended.

File Position

Each `read` or `write` starts at the file position and moves it forward by the number of bytes it copied. `lseek` moves the position without copying anything, and returns the new position:

```
off_t pos = lseek(fd, 0, SEEK_CUR);   // where am I? (move by 0 from here)
lseek(fd, 0, SEEK_SET);               // go to byte 0, the start
lseek(fd, 100, SEEK_CUR);             // go forward 100 bytes from here
off_t size = lseek(fd, 0, SEEK_END);  // go to the end: returns the size
lseek(fd, -4, SEEK_END);              // go to 4 bytes before the end
```

- A `read` that starts at or past the end of the file returns 0.
- `lseek` may move past the end. A `write` there makes the file longer, and the skipped bytes read back as zeros.

open() Flags

The second argument of `open` is one access mode, combined with `|` with any number of other flags:

Flag	Meaning
`O_RDONLY`, `O_WRONLY`, `O_RDWR`	read only, write only, or both (pick exactly one)
`O_CREAT`	create the file if it does not exist
`O_TRUNC`	if the file exists, cut it to 0 bytes
`O_APPEND`	every `write` goes to the current end of the file
`O_EXCL`	with `O_CREAT`: fail if the file already exists

With `O_CREAT`, a third argument gives the new file's permissions. `0644` is octal: the owner can read and write (6), and the group and everyone else can only read (4, 4).

The Shell's Redirections Are open() Flags

When the shell runs `cmd > path`, it opens `path` before it starts `cmd`. Each redirection is one combination of the flags:

```
open(path, O_WRONLY | O_CREAT | O_TRUNC, 0644);    // cmd >  path
open(path, O_WRONLY | O_CREAT | O_APPEND, 0644);   // cmd >> path
open(path, O_RDONLY);                              // cmd <  path
```

`>` uses `O_TRUNC`, so an existing file loses its old contents. `>>` uses `O_APPEND`, so the old contents stay and the new output goes after them. The shell then makes the new descriptor the command's standard output (for `<`, its standard input) before the command starts.

File Metadata: stat

`stat` reads a file's **metadata**, the facts the kernel keeps about the file, without reading any of its bytes. `fstat(fd, &st)` does the same for an open descriptor.

```
struct stat st;
if (stat("data.txt", &st) < 0) { perror("data.txt"); return 1; }

st.st_size            // the size in bytes
st.st_mode            // the type (file, directory, ...) and the permission bits
st.st_ino             // the inode number, which names the file on its disk
st.st_blocks          // the disk space used, in 512-byte units
S_ISREG(st.st_mode)   // true for a regular file
S_ISDIR(st.st_mode)   // true for a directory
```

The field types differ between systems, so cast before printing: `printf("%lld\n", (long long)st.st_size)`. `04-stat.c` prints all four fields.

Buffered vs. Unbuffered I/O

Writing one byte per `write` call makes one system call per byte. Collecting the bytes in an array and writing the array when it is full makes one system call per 4096 bytes:

```
// Unbuffered: 1,000,000 system calls
for (int i = 0; i < 1000000; i++)
    write(fd, &c, 1);

// Buffered: 245 system calls
char buf[4096];
int pos = 0;
for (int i = 0; i < 1000000; i++) {
    buf[pos++] = c;
    if (pos == sizeof buf) {          // the buffer is full: write it out
        write(fd, buf, pos);
        pos = 0;
    }
}
write(fd, buf, pos);                  // write out the last, partial buffer
```

Run It: One Byte per System Call

The C library's `stdio` functions (`printf`, `fputc`, `fwrite`, `fgets`) do this for you. A `FILE *` holds a buffer, and `stdio` calls `write` only when the buffer is full, when you call `fflush`, or when you call `fclose`. `stdout` to a terminal is the exception: it is written out at every newline.

`03-buffering.c` writes 1,000,000 bytes to a file the two ways above, and a third way with `fputc`:

```
$ ./03-buffering                                  # Linux, Intel i5-6300HQ
write, 1 byte per call:     1000000 calls     1.552 s
write, 4096 bytes per call:     245 calls     0.002 s
fputc, 1 byte per call:                      0.005 s
$ strace -c -e trace=write ./03-buffering > /dev/null    # columns shortened
  calls  syscall
1000491  write               # 1,000,000 + 245 + 245 from fputc + 1 printf
```

The bytes written are the same. Buffering is about 800 times faster because it makes 4,000 times fewer system calls. `fputc` makes the same 245 calls as our own buffer, plus a little more work per byte inside the C library.

On a Mac the same program prints 2.3 s, 0.001 s and 0.036 s.

AI Systems Connection

A training job reads its dataset again on every pass, and the GPU sits idle whenever the next batch has not arrived. Moving data from disk to GPU is a chain of today's ideas: the data loader calls `read`, the kernel copies bytes from the disk into its **page cache** (copies of recently read file blocks, kept in RAM), then into the loader's buffer, and the loader sends the buffer to the GPU.

Two of today's rules decide how fast that chain runs:

- **Bytes per system call.** Datasets are packed into a few large files (formats such as WebDataset, TFRecord or Parquet) instead of millions of small image files. One large sequential read replaces thousands of `open`, `read` and `close` calls.
- **Who has the position.** Loader processes often open a file once and then `fork`. The child gets a copy of the descriptor table that points to the same entries, so all the workers share one file position. `pread(fd, buf, n, offset)` reads at the offset it is given and leaves the shared position alone, so one worker's reads do not move another's.

The Lecture in One Slide

- The kernel keeps a numbered list of open files for each process. A file descriptor is a number in that list: `read(3, ...)` reads from whatever is open at number 3.
- Numbers 0, 1 and 2 (input, output, errors) are open when the program starts. `open` uses the smallest free number, so the first `open` returns 3.
- Each `open` call also makes a record that holds the file position. Two descriptors that point to the same record move the same position.
- `read` returns how many bytes it copied, 0 at the end of the file, or -1 on an error. It can copy fewer bytes than you asked for, so call it in a loop.
- If a signal arrives while `read` is waiting, `read` returns -1 with `errno` set to `EINTR`. Nothing went wrong and nothing was read, so call it again.
- `lseek` moves the position. The `open` flags say how the file is opened and what happens to a file that already exists.
- `stat` gives a file's size, type and permissions without reading the file.
- Every system call is expensive, so write 4096 bytes per call, not one, or let `stdio` collect the bytes and call `fflush` when they must go out now.

Practice

1. Write `mycat`, which copies standard input to standard output using only `read` and `write`. What happens if your loop stops at the first short read?
2. A process has descriptors 0, 1 and 2 open. It opens `a.txt`, opens `b.txt`, closes the first one, and opens `c.txt`. Which descriptor does `c.txt` get?
3. A file holds the 7 bytes `abcdefg`. The program opens it, calls `read(fd, buf, 3)` three times, then calls `lseek(fd, 0, SEEK_CUR)`. What does each call return?
4. What does `lseek` do on a pipe? Why?
5. Which `open` flags does the shell use for `cmd >> log.txt`? Why is `O_APPEND` safer than `lseek(fd, 0, SEEK_END)` followed by `write` when two processes write to the same log?

Answers to Practice

1. The loop must stop only when `read` returns 0 (end of input) or -1 (an error). If it stops at the first short read, it stops after the first line when typing at a terminal, and after the first chunk from a pipe.
2. Descriptor 3. `a.txt` gets 3 and `b.txt` gets 4. Closing 3 frees it, and `open` returns the lowest unused number.
3. 3, 3, then 1 (only `g` is left, a short read), and `lseek` returns 7. A fourth `read` would return 0.
4. It fails with `errno` set to `ESPIPE` ("Illegal seek"). A pipe has no position: bytes leave it as they are read, so there is nothing to seek back to.

5. `O_WRONLY | O_CREAT | O_APPEND`. With `lseek` then `write`, the other process can write between the two calls, and this process then overwrites what it wrote. With `O_APPEND`, the kernel moves to the end and writes in one step, so no write can land in between.

Reading

- CS:APP §10.1–10.5 (System-Level I/O)
- `man 2 open`, `man 2 read`, `man 2 lseek`, `man 2 stat`
- The sample code in `sample code/lecture12/`